

Lab Report: Lab 8 - Deployment View & Quality Attribute Analysis (ATAM)

Contents

1. Cover Page	1
2. Abstract/Summary	1
3. Lab Specific Section: Deployment and Quality Analysis	2
4. Conclusion for the 8-Lab Series	3

1. Cover Page

- **Title:** Deployment View and Architecture Trade-off Analysis Method (ATAM) for OmniDash
- **Course Info:** Software Architecture
- **Student Details:** Nguyễn Văn Trường, 23010371
- **Date:** June 5, 2026

2. Abstract/Summary

This final lab shifts the focus from software implementation to physical deployment and architectural evaluation. The objective was to model the physical execution environment of the OmniDash Microservices Architecture and rigorously evaluate its structural decisions against critical Non-Functional Requirements (NFRs). We constructed a UML Deployment Diagram to visualize the distribution of containers, networks, and databases across distinct nodes. Subsequently, a simplified Architecture Trade-off Analysis Method (ATAM) was conducted, focusing on two primary quality attributes: Scalability and Availability. By simulating high-stress scenarios (a 10x traffic spike and a complete subsystem failure), we systematically compared the current Distributed Microservices architecture against the original Monolithic Layered design from Lab 3. The analysis effectively documented the inherent trade-offs, providing a comprehensive justification for the distributed design pattern utilized in the OmniDash platform.

3. Lab Specific Section: Deployment and Quality Analysis

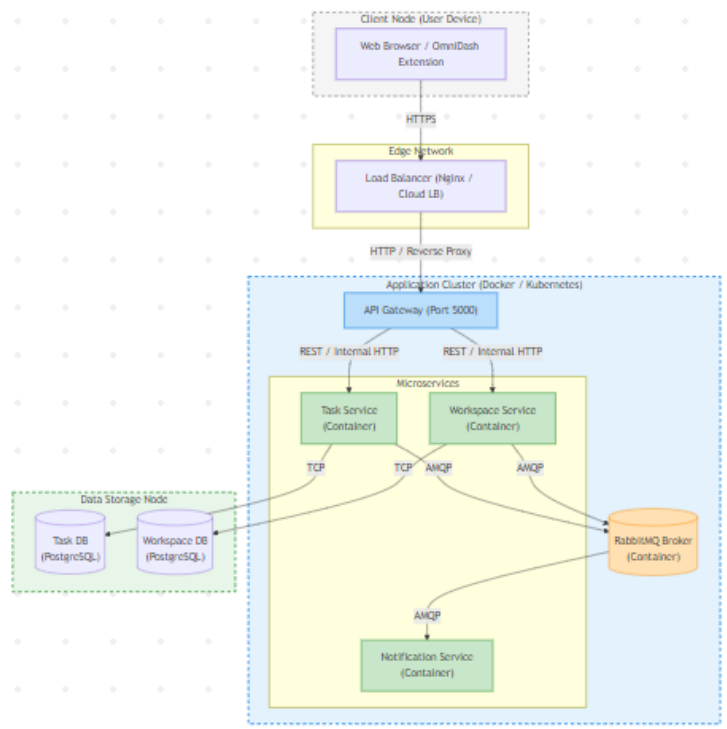
This section documents the physical deployment strategy and the analytical evaluation of the OmniDash system's architecture.

3.1 UML Deployment View

While previous C4 and Component diagrams modeled the logical structure of the software, the Deployment View illustrates how these software artifacts are mapped to physical or virtual hardware resources (Nodes).

To support a distributed ecosystem, the OmniDash deployment relies heavily on containerization (e.g., Docker).

- **Client Node:** Represents the end-user's execution environment, running the Next.js Web App or the Manifest V3 Browser Extension, communicating outward via HTTPS.
- **Edge Network Node:** A Load Balancer (such as Nginx) acts as the initial physical entry point, distributing incoming HTTPS traffic across available API Gateway instances.
- **Application Cluster Node:** Represents the virtualized execution environment (e.g., Kubernetes Pods or distinct Docker containers). It physically isolates the API Gateway, the individual microservices (Task, Workspace, Notification), and the RabbitMQ Message Broker.
- **Data Storage Node:** Represents isolated database servers. Crucially, the diagram illustrates the "Database-per-Service" constraint in a physical sense, showing separate database instances for Tasks and Workspaces, communicating via TCP protocols.



3.2 Architectural Trade-off Statement

Based on the ATAM analysis, transitioning OmniDash from a Layered Monolith to an Event-Driven Microservices ecosystem yields significant advantages but introduces unavoidable compromises.

- **The Benefit:** The distributed architecture provides superior **Scalability** (the ability to independently scale bottlenecked services) and maximum **Fault Isolation / Availability** (the failure of an auxiliary service like Notifications does not cripple the entire platform).
- **The Cost (Trade-off):** This resilience comes at the cost of immense **Operational Complexity**. Deploying OmniDash now requires managing Docker containers, orchestrating a Load Balancer, maintaining an API Gateway, and monitoring an external Message Broker (RabbitMQ). Furthermore, debugging becomes inherently more difficult, as a single user request traverses multiple network boundaries before completion.

Ultimately, given OmniDash's goal to support a growing user base of students and professionals, the trade-off of increased complexity is justified by the requirement for high availability and rapid scalability.

4. Conclusion for the 8-Lab Series

The completion of this 8-lab series marks the successful architectural evolution of the OmniDash platform. We began with formal Requirements Elicitation (Lab 1) to identify the critical Architecturally Significant Requirements (ASRs). Initially, we adopted a Layered Monolith pattern (Labs 2 & 3) to establish a strong foundation of separation of concerns.

Recognizing the limitations of a monolith regarding scalability and fault tolerance, we logically decomposed the system (Lab 4) and implemented independent microservices with strict data isolation (Lab 5). To secure and route external traffic, an API Gateway was engineered (Lab 6). To decouple heavy background processes, we successfully integrated an Event-Driven Architecture utilizing RabbitMQ (Lab 7). Finally, this lab (Lab 8) visualized the physical deployment and formally validated our architectural journey through the ATAM process. The resulting OmniDash system is a robust, distributed, and highly resilient platform, successfully applying the core principles of modern software architecture.